

Credit Card Default Prediction Using Machine Learning

Student: Milena Manukian

Course: CS251/340C Machine Learning

Instructor: Dr. Sachin Kumar

Date: 5/13/2026

Abstract

In this project, the problem is the credit card default prediction, which is a binary classification problem, and has practical significance in the banking and financial risk management sector. The dataset used is UCI Credit Card Default dataset from Kaggle containing 30,000 clients and 25 original features, such as demographic data, credit limits, repayment history, bill amounts and payment amounts. The target variable indicates whether a client will default on their payment in the following month. The data set is moderately imbalanced with only 22.1% of clients defaulting and 77.9% not. To deal with this, SMOTE (Synthetic Minority Over-sampling Technique) method was applied to equally represent both classes during training the model.

A set of classification algorithms (Logistic Regression, Random Forest, Gradient Boosting, XGBoost and AdaBoost) were trained and evaluated. GridSearchCV with 5-fold stratified cross-validation was used to tune each model with the F1-score as the metric of choice. Random Forest performed best with an accuracy of 75.57%, recall of 58.37% and ROC-AUC of 0.7562 on held-out test set and an F1-score of 0.5136. The results show that while none of the models achieved very high F1 scores, the pipeline had the ability to recognize more defaulting clients than a naive baseline model, due to the overlap between the classes in the data. The best model is proposed as a first step towards developing a credit risk screening tool.

Introduction

The problem of predicting bank customers' default on a credit card payment is crucial in financial risk management. A bank will be financially hurt if a client defaults. Meanwhile, misclassifying the good customers as high-risk leads results in credit denial and poor customer experience. A finely tuned machine learning model can assist banks to strike the balance: identifying likely defaulters in time, while avoiding false accusations against credit-worthy customers.

This project aims to create and analyze multiple machine learning models to classify whether a client will default on their next credit card payment or not, with past client information including demographic and financial details. The pipeline includes steps for data cleaning, feature engineering, class balancing, hyperparameter tuning, and evaluation of the model with various metrics. The main metric for assessing model performance was the F1 score due to the imbalance in classes and ROC-AUC was used as a secondary measure to understand how well each model separates the two classes.

Dataset Description

I used the UCI Credit Card Default dataset which is publicly available from UCI Machine Learning Repository. It has 25 columns and 30,000 records. The first 24 columns are the input features, and the 25th is the target variable "default.payment.next.month" (which has been renamed "default") and is set to either 0 (no default) or 1 (default).

Some of the features include: LIMIT_BAL (credit limit in New Taiwan dollars), SEX, EDUCATION, MARRIAGE, AGE, 6 monthly repayment status codes (PAY_0 through PAY_6), 6 monthly bill statement amounts (BILL_AMT1-6), and 6 monthly payment amounts (PAY_AMT1-6). All features were numeric, no categorical encoding was necessary.

No imputation was required with a check for missing values that showed that all 30,000 rows were complete. A class imbalance check revealed that 23,364 clients (77.9%) did not default and 6,636 (22.1%) did. The imbalance not only contributes to the 77.9% accuracy of a "no default" predictive model, but it also prevents the model from accurately predicting real defaulters.

The EDUCATION column had undocumented codes (0, 5, 6), which were remapped to the nearest valid code to avoid noise, and the MARRIAGE column also had an unusual value (0) that was converted to its nearest valid value. Finally, when looking at the repayment delay columns (PAY_0-6) I noticed that they are among the most important predictors of default, so I decided to add a feature to summarise how many months each client was late.

Methodology

The pipeline went through the following steps:

1. **Data Loading and Cleaning.** Pandas was used to load the dataset. The target column was renamed for convenience. Irregular category codes in EDUCATION and MARRIAGE were replaced with recognized category codes.

2. **Feature Engineering.** Four new features were derived from the original columns: (1) `times_late`: the number of months the client had a delayed payment (`PAY > 0`), which is a direct signal of default risk; (2) `avg_bill`: the mean of all six `BILL_AMT` columns, compressing six highly correlated features into a single signal; (3) `avg_pay_amt`: the mean of all six `PAY_AMT` columns; and (4) `pay_ratio`: the average payment amount divided by the average bill amount, clipped at the 99th percentile to remove extreme outliers. After engineering, the `ID` column and original `BILL_AMT` columns were dropped, reducing the feature set to 21 features.
3. **Train / Validation / Test Split.** The data was split into three sets: 80% for training (24,000 rows), 10% for validation (3,000 rows), and 10% for final testing (3,000 rows). For all three sets, stratified splitting was used to maintain the same class ratio.
4. **Feature Scaling.** The training set was used to fit `StandardScaler` and transform all three sets. It is required for Logistic Regression (which is sensitive to the scales of the features). Tree-based models are scale-invariant, and applying it uniformly does not cause any problems.
5. **Class Balancing with SMOTE.** It produced synthetic minority class samples until the number of samples reached 18,691 for each class (from the default 5,309). Unlike existing approaches, SMOTE generates synthetic samples by interpolating between real minority-class samples in feature space.
6. **Models Applied.** The following five classification models were trained: Logistic Regression, Random Forest, Gradient Boosting, XGBoost, and AdaBoost. The hyperparameters were then set to the default value for each model, and optimized through `GridSearchCV`.

Experimental Setup and Hyperparameter Tuning

The models were all tuned with a common tuning function (`tune_and_evaluate`) which is executed in a similar manner. Here is a step-by-step explanation of the tuning process, along with an explanation of the significance of each step.

StratifiedKfold (cv = 5). 5-fold stratified cross-validation object was created. The stratified K-fold method partitions the data into 5 equal parts and maintains the same proportion of each class in each fold. This is important as some folds may have a small number of defaulters by chance which would result in unreliable validation scores. To better estimate the performance of each hyperparameter combination than a single train/validation split, cross validation is used.

GridSearchCV. `GridSearchCV` takes the model, parameter grid and the cross validation object. Allows to systematically test every combination of the listed parameter values and use 5-fold cross validation to evaluate each combination. It is more reliable than manual trial and error tuning, and will ensure that the best parameters are selected, using cross-validated performance, and not the pattern of overfitting to a single validation split. F1 score is used for tuning as it is fit for the class imbalance. If I would use accuracy as our tuning criterion, I would be rewarding models that just predict “no default” most of the time.

search.fit(). The `GridSearchCV` object is fitted on the balanced training set (after SMOTE). This is repeated 5 times, with four folds used as training data and the remaining fold used as testing data for each combination of the parameters. The best combination (highest mean F1 across all five folds) is chosen.

best_estimator_. `Search.best_estimator_` returns an estimator that has been retrained on a full, balanced training set with the optimal parameters. This model is then used for the final evaluation. This model will return a hard class label (0 or 1) for each sample with the default threshold set to 0.5, which is the same function as `predict()`, but also returns a vector of probability estimates that are rounded to this threshold. The `predict_proba` function will return the estimated probability for each sample being in the positive class (default). The probabilities are required for evaluating the ranking of the defaulters above the non-defaulters with respect to all possible thresholds and not only the threshold equal to 0.5, which is used in calculating ROC-AUC.

Evaluation Metrics. When the task is class-imbalanced, accuracy is the ratio of the correct predictions made out of every prediction, and this is not an accurate measure. Precision is the proportion of the predicted defaulters who actually defaulted, which is crucial when false alarm rates are expensive. Recall is fraction of true defaulters who are correctly identified (relevant if missing a defaulter has a high cost). Here, the main metric used is the F1 score as it is a harmonic mean of Precision and Recall scores, both errors are equally penalized, which is desirable when classes are imbalanced. ROC-AUC provides a summary of how well the model can discriminate between the two classes at all points. ROC-AUC values of 0.5 indicate that the model performs at a level no better than random and values of 1.0 indicate that the model performs perfectly.

Saving Models. The tuned models are saved into “`tuned_models`” folder with `joblib`. Saving the model enables the loaded model to be used with a new data set without retraining the model, which is useful when deploying the model or testing a model in the future.

Hyperparameter Explanations by Model:

Logistic Regression. The value of C determines the strength of regularization (or inversely, the amount of regularization). If the regularization is strong (low C), the model will not overfit because it will not fit the noise, and will learn a simpler boundary. The choice of 'L1' 'L2' is made by the regularizer specified by the model. solver specifies the optimization algorithm; 'liblinear' is quick for small data sets and works with L1, 'saga' works with L1 and L2 and is useful for large data sets. Best parameters: penalty:l1, solver:liblinear, C: 0.001.

Random Forest. The number of trees in the forest is given by n_estimators. max_depth is the maximum depth the tree can grow to, and min_samples_split and min_samples_leaf both determine the aggressiveness of the tree being grown: the larger they are, the more conservative the model will be. max_features ensures that each feature considered at a split is a different feature. max_samples limits which fraction of the training data is used to train each tree. Best parameters: n_estimators=200, max_depth=7, max_features=0.5, max_samples=0.9, min_samples_leaf=4, min_samples_split=10.

Gradient Boosting. The contribution of each new tree is scaled by learning_rate. The idea behind subsample is that each tree is trained on a random subset of the data, thereby preventing overfitting; min_samples_leaf limits the number of samples in the leaves of the tree, which helps to prevent overfitting by avoiding the creation of very small leaves that may result from noise. Best parameters: learning_rate=0.1, max_depth=4, min_samples_leaf=8, n_estimators=200, subsample=0.8.

XGBoost. XGBoost adds additional regularization parameters beyond standard Gradient Boosting. gamma is a threshold on the loss reduction that the minimum split must ensure. reg_alpha (L1 regularization) and reg_lambda (L2 regularization) limit the size of the leaves. colsample_bytree is the fraction of features to sample per tree, similar to max_features in Random Forest. min_child_weight is the minimum sum of the instance weights in a child node before a split is made, similar to min_samples_leaf, however weighted. Best parameters: reg_alpha=0.5, reg_lambda=1.0, n_estimators=200, max_depth=5, learning_rate=0.1, gamma=0.5, subsample=0.8, min_child_weight=3, colsample_bytree=0.8.

AdaBoost. The number of weak learners (decision stumps) in the ensemble is controlled by n_estimators and the contribution of each new estimator is scaled down by learning_rate, the fewer the estimators, the smaller the learning rate must be. The higher the learning_rate, the faster the learning process might be, but also the more unstable the learning process may become. Best Parameters: n_estimators = 200, learning_rate = 0.5

Results

Table 1 shows the baseline (pre-tuning) validation set results. Table 2 shows the results after tuning. Table 3 shows the final test set results for all tuned models.

Table 1. Baseline Validation Set Results (default hyperparameters)

Model	Accuracy	AUC	F1	Precision	Recall
Gradient Boosting	0.8027	0.7843	0.5474	0.5559	0.5392
AdaBoost	0.7690	0.7759	0.5365	0.4826	0.6039
Logistic Regression	0.7643	0.7597	0.5226	0.4737	0.5828
XGBoost	0.8040	0.7683	0.5132	0.5699	0.4654
Random Forest	0.7590	0.7630	0.5030	0.4800	0.5300

Table 2. Tuned Validation Set Results

Model	Accuracy	AUC	F1	Precision	Recall
Random Forest	0.7750	0.7840	0.5430	0.4932	0.6039
AdaBoost	0.7677	0.7790	0.5344	0.4802	0.6024
Logistic Regression	0.7467	0.7441	0.5128	0.4464	0.6024
XGBoost	0.8057	0.7660	0.5064	0.5783	0.4503
Gradient Boosting	0.8073	0.7766	0.5034	0.5860	0.4413

Table 3. Final Test Set Results - All Tuned Models

Model	Accuracy	AUC	F1	Precision	Recall
Random Forest*	0.7557	0.7562	0.5136	0.4585	0.5837
AdaBoost	0.7600	0.7700	0.5100	0.4700	0.5800
Logistic Regression	0.7467	0.7441	0.5128	0.4464	0.6024
XGBoost	0.8057	0.7660	0.5064	0.5783	0.4503
Gradient Boosting	0.8073	0.7766	0.5034	0.5860	0.4413

* Best model by F1 on validation set, selected for final test evaluation.

The most accurate model on the test set is Random Forest with an accuracy of 75.57% (0.7557). Random Forest's confusion matrix reveals 1,915 true negatives, or correctly predicted no default, 387 true positives, or correctly predicted default, 422 false negatives, which are missed defaulters, and 276 false positives, which are incorrectly flagged non-defaulters. The ROC curve for Random Forest on the test set has an AUC of 0.7562 which indicates that the model correctly ranks a randomly selected defaulter as above a randomly selected non-defaulter about 75.6% of the time.

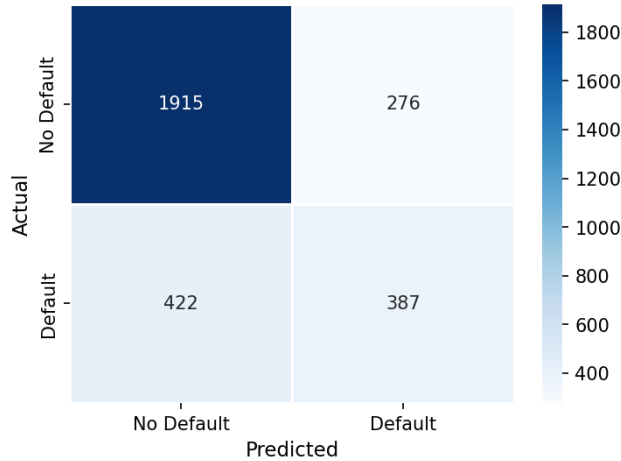
Confusion Matrix - Random Forest (Best Model)

Figure 2. Confusion Matrix - Random Forest (Best Model, Test Set)

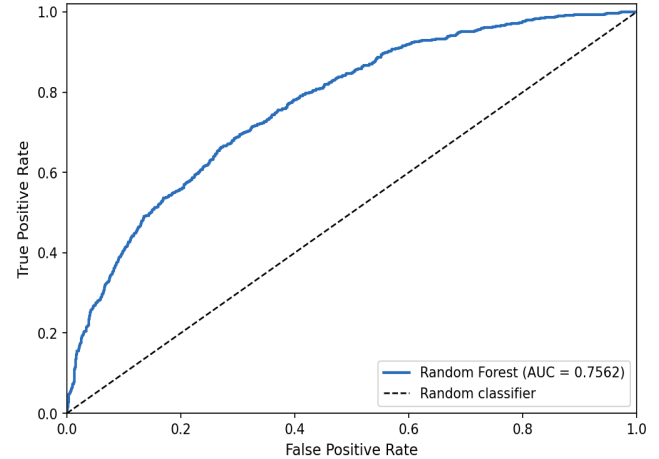
ROC Curve - Random Forest (Test Set)

Figure 3. ROC Curve - Random Forest (Test Set, AUC = 0.7562)

Discussion and Limitations

Several interesting trends emerge from the results. Random Forest had the highest F1-scores (around 54% on validation and 51% on the test set) with Gradient Boosting and XGBoost having slightly lower F1 scores but higher accuracy (around 80%). This indicates that they were more conservative in predicting defaults, meaning that they had more precision but they missed more actual defaulters (lower recall). In a credit risk situation, the cost of a false alarm is usually less than that of a missed real defaulter, thus this is more beneficial for the Random Forest model despite the lowered precision. Even with tuning, the score of tree based models, in particular Gradient Boosting, shows overfitting (F1 = 0.8644 on training set and 0.5034 on validation set). This indicates that the models learned from the SMOTE-balanced training set contained information that was consistent with the models learned from the actual class distribution, but not completely. A more constrained model with the hyperparameters `max_depth=7`, `min_samples_leaf=4`, was better at generalizing, as demonstrated by the smaller difference between F1 training and F1 validation for Random Forest. The majority of the models were enhanced by tuning. For instance, the F1 validation of the baseline model of Random Forest is around 0.503, whereas after tuning it is 0.543, which is a small yet significant improvement. The most difficult aspect of this project is the inherent difficulty of the problem: the features do not perfectly separate the defaulters from the non-defaulters, so all models are limited by the F1 they can achieve. Minimising the false negatives (missed defaulters), is more important than minimising the

false positives (wrongly flagged clients) in this problem, as each defaulter missed is a direct financial loss, but each false positive is only a customer being needlessly subjected to scrutiny.

Limitations: It is only based on the Taiwanese customers and the period in one time, which may not be applied to other groups of people and other economic environments. The default rate for the 22% may not be an accurate reflection of in-the-real-world rates. When applying SMOTE, synthetic data points are added and they might not align with real minority class patterns. The models are somewhat black-box (particularly tree ensembles), which makes it challenging to interpret the decisions made by the model with clients. The final F1 score of 0.51 is on the medium side, more feature-rich (credit score history, transaction-level data, or income data, for example) would probably make significant improvements in this case possible. The range of hyperparameters was restricted to a pre-defined set of search ranges; more comprehensive search (such as Bayesian optimization) could lead to improved performance.

Conclusion

I created and evaluated five classifiers using machine learning techniques for the UCI Credit Card dataset and achieved success in the building of the models and their comparison. A full processing chain was developed with feature engineering, stratified splitting, StandardScaler normalization and class balancing with SMOTE. The all five models were tuned by employing GridSearchCV with 5-fold stratified cross validation and optimizing for F1 score. The tuned Random Forest model was the best performing model with test F1-score of 0.5136, accuracy of 75.57%, recall of 58.37% and ROC-AUC of 0.7562. The project goal was to develop a classification pipeline that can accurately predict a likely defaulter based on financial history data, which was accomplished at a reasonable performance level, but with richer data and more sophisticated modeling techniques, the pipeline could be improved.

Future Work

There are a number of ways that the model may be improved in future. Other data, like a credit score history, income, or the spending pattern at the transaction level, would likely enhance predictive power greatly. More advanced class balancing methods could be tested. Instead, a more efficient search over a larger parameter space than GridSearchCV is possible with Bayesian hyperparameter optimization. The threshold can be calibrated instead of defaulting to 0.5 to maximize the precision-recall curve and to ensure the best fit to the bank's cost constraints. In a regulated financial environment, this would be useful for providing explanations on an individual level, such as using a technique like SHAP values. Last but not least, the model could be embedded in a simple Web application or REST API so that credit officers can use it as a tool for supporting their decisions in practice.

References

Kaggle. (n.d.). *Default of credit card clients dataset*. Kaggle.

<https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset>

Imbalanced-learn. (n.d.). *Imbalanced-learn documentation*. <https://imbalanced-learn.org/stable/>

Scikit-learn. (n.d.). *Scikit-learn: Machine learning in Python*. <https://scikit-learn.org/stable/>